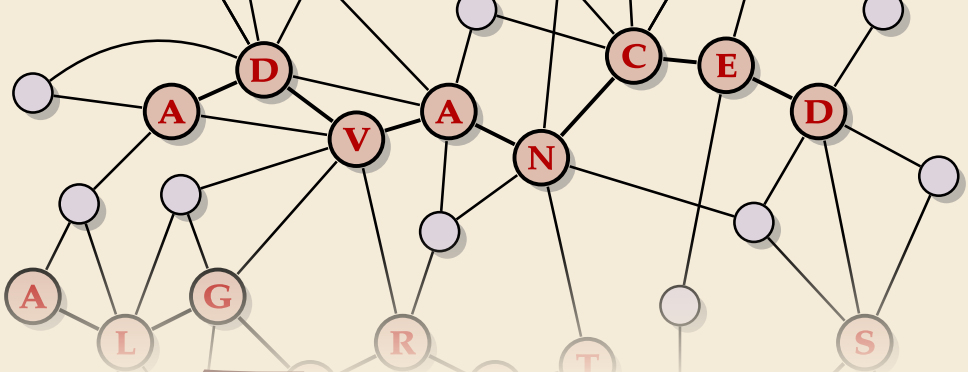
$f(k) \cdot n^{O(1)}$

Problems w/ FPT Algorithms

- p- Vertex Cover
- p- MaxSAT
- p- Planar Independent Set
- p- Closest String

- p- variables-SAT



5 Parameterized Hardness

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-11-12 14:12`

Outline

5 Parameterized Hardness

5.1 Parameterized Reductions

5.2 Nondeterministic FPT: Para-NP

5.3 Bounded Nondeterminism: W[P]

5.4 W[1]-Hardness

5.5 W[1]-Hard Problems

How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time

How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time
- example: p-Clique

$\rightsquigarrow$ maybe no fpt algorithm can exist for p-Clique!

$O(n^k)$ brute force

How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time
- example: p-Clique

$\rightsquigarrow$ maybe no fpt algorithm can exist for p-Clique!

- **Problem:** Certainly exists in case P = NP

$\rightsquigarrow$ strongest lower bound we can hope for will have to be conditional on P $\neq$ NP

How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time
- example: p-Clique

$\rightsquigarrow$ maybe no fpt algorithm can exist for p-Clique!

- **Problem:** Certainly exists in case $\mathsf{P} = \mathsf{NP}$

$\rightsquigarrow$ strongest lower bound we can hope for will have to be conditional on $\mathsf{P} \neq \mathsf{NP}$

- Typical complexity-theory results:
  No algorithm has property X unless (more of less widely believed) complexity hypothesis Y fails.

5.1 Parameterized Reductions

FPT Reductions

Goal: Compare relative hardness of *parameterized* problems

⇝ Need a notion of reductions on parameterized problems

- to preserve (non)existence of fpt algorithms, need to keep small k

FPT Reductions

Goal: Compare relative hardness of *parameterized* problems

⇝ Need a notion of reductions on parameterized problems

- to preserve (non)existence of fpt algorithms, need to keep small k

Definition 5.1 (Parameterized Reduction)

Let (Π_1, κ_1) and (Π_2, κ_2) be two parameterized problems, $\Pi_i = (L_i, Y_i, \Sigma_i)$.
An *fpt-reduction* (*fpt many-one reduction*) from (Π_1, κ_1) to (Π_2, κ_2) is a mapping $A : L_1 \to L_2$ so that for all $x \in L_1$

$n_1 = |x|$, $k_1 = \kappa_1(x)$

1. (*equivalence*) $x \in Y_1 \iff A(x) \in Y_2$,

A running time $O(f(k_1) \cdot n_1^{\alpha(1)})$

2. (*fpt*) A is computable by an fpt-algorithm (w.r.t. κ_1), and
3. (*parameter-preserving*) $\kappa_2\big(A(x)\big) \leq g\big(\kappa_1(x)\big)$ for a computable function $g : \mathbb{N} \to \mathbb{N}$.

We then write $(\Pi_1, \kappa_1) \leq_{fpt} (\Pi_2, \kappa_2)$. ◀

Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

Recall:

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : \big(u \in V' \vee v \in V'\big)$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

Recall:

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : \big(u \in V' \vee v \in V'\big)$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

- We know: IndependentSet $\leq_p$ VertexCover:
  - Set $G' = G$ and $k' = |V(G)| - k$ (Complement of an indep. set must be a vertex cover, and vice versa!)

Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

Recall:

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : \big(u \in V' \vee v \in V'\big)$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

- We know: IndependentSet $\leq_p$ VertexCover:
  - Set $G' = G$ and $k' = |V(G)| - k$ (Complement of an indep. set must be a vertex cover, and vice versa!)
- $\not\Rightarrow$ p-IndependentSet $\leq_{fpt}$ p-VertexCover
  - Indeed, we know VertexCover $\in$ FPT, but IndependentSet probably isn't.

Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

Recall:

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : (u \in V' \vee v \in V')$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

- We know: IndependentSet $\leq_p$ VertexCover:
  - Set $G' = G$ and $k' = |V(G)| - k$ (Complement of an indep. set must be a vertex cover, and vice versa!)
- $\not\Rightarrow$ p-IndependentSet $\leq_{fpt}$ p-VertexCover
  - Indeed, we know VertexCover $\in$ FPT, but IndependentSet probably isn't.
- But: p-IndependentSet $\leq_{fpt}$ p-Clique (and p-Clique $\leq_{fpt}$ p-IndependentSet)
  - Set $G' = (V, \binom{V}{2} \setminus E)$ and $k' = k$ (Independent set iff clique in complement graph)

5.2 Nondeterministic FPT: Para-NP

Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

Definition 5.2 (para-NP)

The class para-NP consists of all parameterized decision problems that are solved by a *non-deterministic* fpt-algorithm. ◀

Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

Definition 5.2 (para-NP)

The class para-NP consists of all parameterized decision problems that are solved by a *non-deterministic* fpt-algorithm. ◀

Some nice properties:

1. para-NP is closed under fpt-reductions.
2. FPT = para-NP $\iff$ P = NP
3. an analogue for *kernalization* in FPT holds for para-NP

non-trivial

Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

Definition 5.2 (para-NP)

The class para-NP consists of all parameterized decision problems that are solved by a *non-deterministic* fpt-algorithm. ◀

Some nice properties:

1. para-NP is closed under fpt-reductions.
2. $\mathsf{FPT} = \text{para-NP} \iff \mathsf{P} = \mathsf{NP}$
3. an analogue for *kernalization* in FPT holds for para-NP

Can define para-NP-hard and para-NP-complete similarly as for NP:

Definition 5.3 (para-NP-hard)

(Π, κ) is para-NP-hard if $(\Pi', \kappa') \leq_{fpt} (\Pi, \kappa)$ for all $(\Pi', \kappa') \in$ para-NP. ◀

Hello hardness, my old friend

Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)

Let (Π, κ) be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ (Π restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

→ rules out that p-Clique para-NP-complete!

because n^k is poly. for $k = O(1)$

Hello hardness, my old friend

Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)
Let (Π, κ) be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ (Π restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

Proof: eventually want $\Pi' \leq_p \Pi_{\kappa \leq d}$

Let $\Pi' = (L', Y', \Sigma')$ be any NP-complete problem and $\kappa_{\text{one}} \equiv 1$.

A nondeterministic polytime algorithm for Π' is a nondeterministic fpt-algorithm for $(\Pi', \kappa_{\text{one}}) \rightsquigarrow (\Pi', \kappa_{\text{one}}) \in$ para-NP.

Hello hardness, my old friend

Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)

Let (Π, κ) be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ (Π restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

Proof:

Let $\Pi' = (L', Y', \Sigma')$ be any NP-complete problem and $\kappa_{\text{one}} \equiv 1$.

A nondeterministic polytime algorithm for Π' is a nondeterministic fpt-algorithm for $(\Pi', \kappa_{\text{one}}) \rightsquigarrow (\Pi', \kappa_{\text{one}}) \in$ para-NP.

(Π, κ) para-NP-hard $\rightsquigarrow$ $(\Pi', \kappa_{\text{one}}) \leq_{fpt} (\Pi, \kappa)$ via some A with

- $x \in Y' \iff A(x) \in Y$,
- running time $f(\kappa_{\text{one}}(x)) \cdot |x|^c = f(1) \cdot |x|^c \rightsquigarrow$ polytime,
- $\kappa(A(x)) \leq g(\kappa_{\text{one}}(x)) = g(1) =: d$.

Hello hardness, my old friend

Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)
Let (Π, κ) be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ (Π restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

Proof:

Let $\Pi' = (L', Y', \Sigma')$ be any NP-complete problem and $\kappa_{\text{one}} \equiv 1$.

A nondeterministic polytime algorithm for Π' is a nondeterministic fpt-algorithm for $(\Pi', \kappa_{\text{one}}) \rightsquigarrow (\Pi', \kappa_{\text{one}}) \in$ para-NP.

(Π, κ) para-NP-hard $\rightsquigarrow$ $(\Pi', \kappa_{\text{one}}) \leq_{fpt} (\Pi, \kappa)$ via some A with

- $x \in Y' \iff A(x) \in Y$,
- running time $f(\kappa_{\text{one}}(x)) \cdot |x|^c = f(1) \cdot |x|^c \rightsquigarrow$ polytime,
- $\kappa(A(x)) \leq g(\kappa_{\text{one}}(x)) = g(1) =: d$.

$\rightsquigarrow$ A is a polytime reduction $\Pi' \leq_p \Pi_{\kappa \leq d} \rightsquigarrow$ $\Pi_{\kappa \leq d}$ is NP-hard. ■

para-NP-complete is too strict

Above Theorem means that many problems cannot be para-NP-complete!

For each of the following

- p-Clique,
- p-IndependentSet
- p-DominatingSet

bounding k by a **constant** d makes *polytime* algorithm possible.

para-NP-complete is too strict

Above Theorem means that many problems cannot be para-NP-complete!

For each of the following

- p-Clique,
- p-IndependentSet
- p-DominatingSet

bounding k by a **constant** d makes *polytime* algorithm possible.

$\rightsquigarrow$ $\Pi_{\kappa \leq d}$ cannot be NP-hard for each of these (unless P = NP)

- but we rather expect them $\notin$ FPT

$\rightsquigarrow$ para-NP theory does not settle complexity status

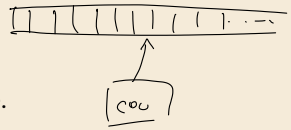
5.3 Bounded Nondeterminism: W[P]

Bye bye, TM

para-NP is too large a class to have interesting complete problems

$\rightsquigarrow$ We must weaken the class. Unfortunately, TM inconvenient here.

Bye bye, TM



para-NP is too large a class to have interesting complete problems
⇝ We must weaken the class. Unfortunately, TM inconvenient here.

≠ word-RAM w-bit numbers

Definition 5.5 (Nondeterministic RAM (NRAM), κ-restricted)
An NRAM M is a (unit-cost) RAM with the additional operation to nondeterministically *guess* a number between 0 and a current register content.

An NRAM M that decides a parameterized problem (Π, κ), $\Pi = (L, Y)$ is *κ-restricted* if on every input $x \in L$ with $n = |x|$ and $k = \kappa(x)$, *every* computation

1. performs at most $f(k) \cdot p(n)$ steps,
2. *at most $g(k)$ of them nondeterministic,*
3. uses at most $f(k) \cdot p(n)$ memory cells, and
4. those never contain numbers larger than $f(k) \cdot p(n)$,

= Markl

for computable functions f and g and a polynomial p.

◀

W[P]

Definition 5.6 (W[P])

The class W[P] is the set of all parameterized problems (Π, κ) decidable by a κ-restricted NRAM. ◀

W[P]

Definition 5.6 (W[P])

The class W[P] is the set of all parameterized problems (Π, κ) decidable by a κ-restricted NRAM. ◀

Example: p-Clique $\in$ W[P]

- brute force does it
  - nondeterministically guess k-set S of vertices k non-det. steps, numbers in $[n]$
  - check if $G[S]$ is a clique
  - $\rightsquigarrow$ $O(n^2 + k + k^2)$ total time

A first W[P]-complete problem?

Define hardness and completeness for W[P] using $\leq_{fpt}$.

What could be the mother of all W[P]-complete problems?

Some parameterized version of SAT? Parameter #variables does not work. (Why?)

Can p-variables-SAT be W[P]-complete?

$\Uparrow$ FPT

non-det guess assignment of variables

$2^k \cdot n^{O(1)}$

A first W[P]-complete problem?

Define hardness and completeness for W[P] using $\leq_{fpt}$.

What could be the mother of all W[P]-complete problems?

Some parameterized version of SAT? Parameter #variables does not work. (Why?)

- What can be guessed using k numbers in $[n]$?

A first W[P]-complete problem?

Define hardness and completeness for W[P] using $\leq_{fpt}$.

What could be the mother of all W[P]-complete problems?

Some parameterized version of SAT? Parameter #variables does not work. (Why?)

- What can be guessed using k numbers in $[n]$?

$\rightsquigarrow$ A subset of variables of *size* k!

Weighted SAT

Definition 5.7 (Weighted Satisfiability)

Given: Boolean formula φ and integer $k \in \mathbb{N}$

Parameter: k

Question: $\exists$ satisfying assignment with weight $= k$?

Recall: weight = #*true* variables

◀

Weighted SAT

for Circuits

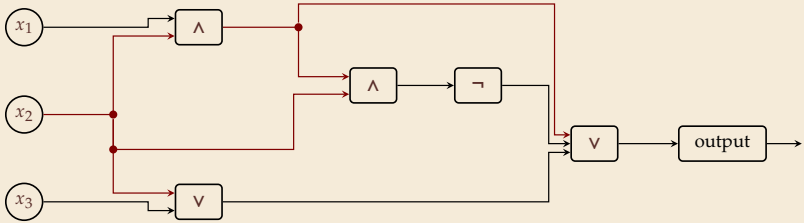
Definition 5.7 (Weighted Satisfiability)

Given: Boolean formula φ and integer $k \in \mathbb{N}$

Parameter: k circuit

Question: $\exists$ satisfying assignment with weight $= k$?

Recall: weight = #*true* variables



a *circuit*: gates may be reused (DAG);
a *formula* is a circuit with all fan-outs 1 (tree)

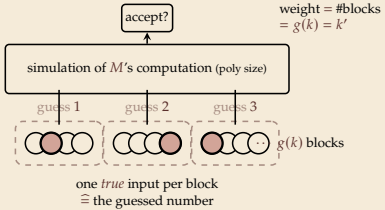
Theorem 5.8 (p-WSAT(CIRC) is W[P]-complete)

The weighted satisfiability problem for boolean **circuits** parameterized by the weight is W[P]-complete.

Weighted SAT – Proof

Proof (sketch):

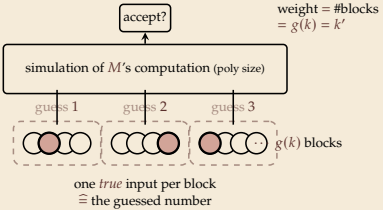
- **"$\in$ W[P]"**: κ-restricted NRAM can guess k indices of *true* inputs (k numbers $\leq n$) and evaluate the circuit bottom-up in polytime ✓



Weighted SAT – Proof

Proof (sketch):

- **"$\in$ W[P]"**: κ-restricted NRAM can guess k indices of *true* inputs (k numbers $\leq n$) and evaluate the circuit bottom-up in polytime ✓
- **"W[P]-hard"**: $(\Pi, \kappa) \in \mathsf{W}[P] \implies (\Pi, \kappa) \leq_{fpt} p\text{-WSAT(CIRC)}$
  - simulate κ-restricted NRAM M on **hardwired** input x as circuit C_x:
  - all computations $\leq f(k)p(n)$ steps, memory cells $\leq \lg\big(f(k)p(n)\big)$ bits
    - $\rightsquigarrow$ deterministic part simulated by circuit of size $\big(f(k)p(n)\big)^{O(1)} = f'(k) \cdot \mathrm{poly}(n)$
  - only $g(k)$ guesses are unknowns: encode in *unary* by $f(k)p(n)$ inputs, **exactly 1** *true*
  - $\rightsquigarrow$ C_x outputs 1 iff M accepts with given guesses



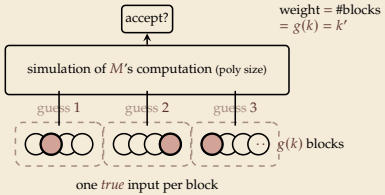
Weighted SAT – Proof

Proof (sketch):

- **"$\in$ W[P]"**: κ-restricted NRAM can guess k indices of *true* inputs (k numbers $\leq n$) and evaluate the circuit bottom-up in polytime ✓
- **"W[P]-hard"**: $(\Pi, \kappa) \in \mathsf{W}[P] \implies (\Pi, \kappa) \leq_{\mathit{fpt}} p\text{-WSAT(CIRC)}$
  - simulate κ-restricted NRAM M on **hardwired** input x as circuit C_x:
  - all computations $\leq f(k)p(n)$ steps, memory cells $\leq \lg\big(f(k)p(n)\big)$ bits
    - $\rightsquigarrow$ deterministic part simulated by circuit of size $\big(f(k)p(n)\big)^{O(1)} = f'(k) \cdot \mathrm{poly}(n)$
  - only $g(k)$ guesses are unknowns: encode in *unary* by $f(k)p(n)$ inputs, **exactly 1** *true*
  - $\rightsquigarrow$ C_x outputs 1 iff M accepts with given guesses

$\rightsquigarrow$ $(x, k) \mapsto (C_x, g(k))$ is an fpt-reduction to p-WSAT(CIRC). ■

Note: *Simulation needs circuits; simulation via formula needs intermediate variables, which increase the weight*



5.4 W[1]-Hardness

Tail-nondeterminism

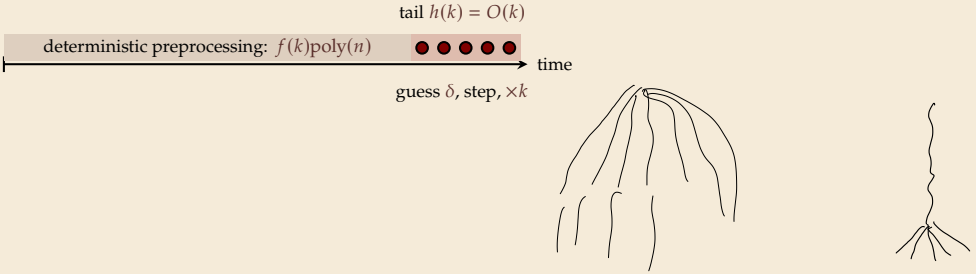
Circuit satisfiability still too strong to show hardness of many interesting problems.
$\rightsquigarrow$ We must weaken the class *further*.

Tail-nondeterminism

Circuit satisfiability still too strong to show hardness of many interesting problems.
$\rightsquigarrow$ We must weaken the class *further*.

Definition 5.9 (tail-nondeterministic NRAM)

A κ-restricted NRAM M for a problem (Π, κ) is called *tail-nondeterministic* if all nondeterministic steps occur only among the last $h(\kappa(x))$ steps. ◀

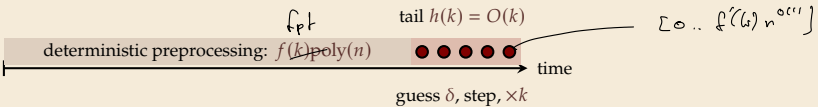


Tail-nondeterminism

Circuit satisfiability still too strong to show hardness of many interesting problems.
$\rightsquigarrow$ We must weaken the class *further*.

Definition 5.9 (tail-nondeterministic NRAM)

A κ-restricted NRAM M for a problem (Π, κ) is called *tail-nondeterministic* if all nondeterministic steps occur only among the last $h(\kappa(x))$ steps. ◀



Definition 5.10 (W[1])

The class W[1] consists of all parameterized decision problems (Π, κ) that are decided by a tail-nondeterministic κ-restricted NRAM. ◀

As before, define hardness and completeness for W[1] w.r.t. $\leq_{fpt}$.

The Parameterized Landscape

$$\mathsf{FPT} \subseteq \mathsf{W}[1] \subseteq \mathsf{W}[2] \subseteq \cdots \subseteq \mathsf{W}[P] \subseteq \mathsf{XP} \cap \mathsf{para\text{-}NP}$$

- XP: problems solvable in time $n^{f(k)}$ ("polytime for every fixed k");
  FPT $\subsetneq$ XP is *provable* (time hierarchy)
- FPT $\subseteq$ W[1]: an fpt-algorithm is a κ-restricted NRAM with 0 guesses
- W[1] $\subseteq$ W[P]: tail-nondeterministic is a special case
- W[P] $\subseteq$ para-NP: a κ-restricted NRAM is a nondeterministic fpt-algorithm
- W[P] $\subseteq$ XP: try all $\leq (f(k)p(n))^{g(k)} = n^{O(g(k))}$ guess sequences

$\rightsquigarrow$ **all inclusions are believed strict**, none is proven except FPT $\subsetneq$ XP.

Hardness for W[1] (or any class above it) is the standard evidence against fpt-algorithms;
the classical support "FPT = W[1] $\Rightarrow$ ETH fails" is void since ETH was refuted (Oct 2026, next unit).

Stop

Definition 5.11 (p-ShortTuringMachineAcceptance)

Given: A nondeterministic (single-tape) Turing machine M, an input x and $k \in \mathbb{N}$ be given.
Parameter: k
Question: Does M accepts x after at most k computation steps? ◀

normally: TM nondet used for guesses

like x_i true or false

guesses only of $|T| \cdot |Q| \cdot 3$

for fixed TM some constant

C^k would be fpt

Stop

Definition 5.11 (p-ShortTuringMachineAcceptance)

Given: A nondeterministic (single-tape) Turing machine M, an input x and $k \in \mathbb{N}$ be given.
Parameter: k
Question: Does M accepts x after at most k computation steps? ◀

- M is part of input, so state space and tape alphabet are not fixed!

⇝ up to n different non-deterministic choices in *each* step. (n is size of encoding of M)

⇝ Trivial algorithm has to simulate up to n^{k+1} steps of M.

- Equivalent here to halting problem for $x = \varepsilon$, since we can hard-wire the given input into the states of a TM M' constructed from M.

p-ShortTuringMachineAcceptance is W[1]-complete

Theorem 5.12

p-ShortTuringMachineAcceptance is W[1]-complete.

Proof (Sketch):

∈ W[1]: Given TM M, input x, number of steps k.
We construct tail-nondeterministic NRAM A to decide if M halts.
A simulated M for k steps (and give its output).
$\rightsquigarrow$ k nondeterministic steps, all within the last $O(k)$ steps.

p-ShortTuringMachineAcceptance is W[1]-complete

Theorem 5.12

p-ShortTuringMachineAcceptance is W[1]-complete. ◀

Proof (Sketch):

∈ W[1]: Given TM M, input x, number of steps k.
We construct tail-nondeterministic NRAM A to decide if M halts.
A simulated M for k steps (and give its output).
$\rightsquigarrow$ k nondeterministic steps, all within the last $O(k)$ steps.

W[1]-hard: $(\Pi, \kappa) \in \mathsf{W}[1]$ $\rightsquigarrow$ $\exists$ tail-nondet. κ-restricted NRAM A, tail length $h(k)$.
On input y: simulate deterministic prefix of $A(y)$, reaching config c.
Tail touches $\leq h(k)$ memory cells, uses $\leq g(k)$ guesses of numbers $\leq f(k)p(n)$.

p-ShortTuringMachineAcceptance is W[1]-complete

Theorem 5.12

p-ShortTuringMachineAcceptance is W[1]-complete. ◀

Proof (Sketch):

∈ W[1]: Given TM M, input x, number of steps k.
We construct tail-nondeterministic NRAM A to decide if M halts.
A simulated M for k steps (and give its output).
$\rightsquigarrow$ k nondeterministic steps, all within the last $O(k)$ steps.

W[1]-hard: $(\Pi, \kappa) \in \mathsf{W}[1]$ $\rightsquigarrow$ $\exists$ tail-nondet. κ-restricted NRAM A, tail length $h(k)$.
On input y: simulate deterministic prefix of $A(y)$, reaching config c.
Tail touches $\leq h(k)$ memory cells, uses $\leq g(k)$ guesses of numbers $\leq f(k)p(n)$.

Build a nondet. TM M_y whose states and tape alphabet encode memory contents; nondeterminism simulates guesses; (y hardcoded into TM)

$\rightsquigarrow$ M_y accepts in $k' = O(h(k))$ steps iff some tail computation of A accepts.
$\rightsquigarrow$ $y \mapsto (M_y, \varepsilon, k')$ is an fpt-reduction. ■

5.5 W[1]-Hard Problems

p-Independent-Set is W[1]-complete

Theorem 5.13

p-ShortTuringMachineAcceptance $\leq_{fpt}$ p-IndependentSet. ◀

Corollary 5.14

p-IndependentSet and p-Clique are W[1]-complete. // see exercise ◀

p-Independent-Set is W[1]-complete

Theorem 5.13

p-ShortTuringMachineAcceptance $\leq_{fpt}$ p-IndependentSet. ◀

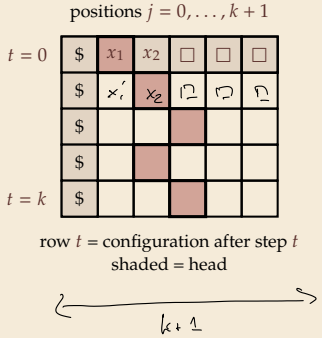
Corollary 5.14

p-IndependentSet and p-Clique are W[1]-complete. ◀

Idea: force IS ≡ one computation

- w.l.o.g. accepting states are absorbing
  - ⇝ ask: is M in an accepting state *after exactly* k steps?
- in k steps the head visits only tape index $0, \ldots, k+1$
  (position 0 holds a left-end marker \$)

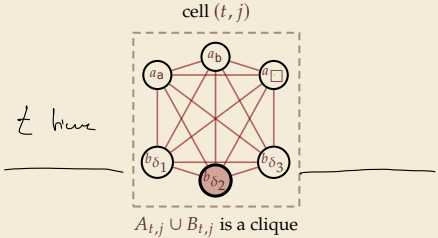
⇝ computation is table with $k+1$ rows (time steps $0..k$) and $k+2$ columns (tape positions)



p-Independent-Set is W[1]-complete – Inside Cells

For every cell (t, j), $0 \le t \le k$, $0 \le j \le k+1$:

- $a_{t,j,\sigma}$ for $\sigma \in \Sigma \cup \{\$, \square\}$
  "after step i, cell j holds σ and the head is not here"
- $b_{t,j,\delta}$ for $\delta = (q, \sigma, q', \sigma', d)$
  "after step i, the head is at j, state q, cell holds σ, and the next TM transition is δ"

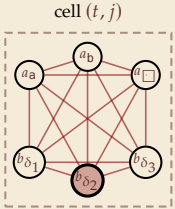


$\rightsquigarrow$ goal: IS must choose one vertex per cell $\rightsquigarrow$ $k' = (k+1)(k+2)$

p-Independent-Set is W[1]-complete – Inside Cells

For every cell (t, j), $0 \le t \le k$, $0 \le j \le k+1$:

- $a_{t,j,\sigma}$ for $\sigma \in \Sigma \cup \{\$, \square\}$
  "after step i, cell j holds σ and the head is not here"
- $b_{t,j,\delta}$ for $\delta = (q, \sigma, q', \sigma', d)$
  "after step i, the head is at j, state q, cell holds σ, and the next TM transition is δ"



$\rightsquigarrow$ goal: IS must choose one vertex per cell $\rightsquigarrow$ $\boxed{k' = (k+1)(k+2)}$

Boundary conditions (delete vertices):

- $t = 0$ (initial config): keep only a_{0,j,x_j} for $j \ge 2$ and $b_{0,1,\delta}$ with $\delta = (q_0, x_1, \circ, \circ, \circ)$
- $j = 0$ (marker): keep only $a_{i,0,\$}$ and $b_{i,0,\delta}$ reading \$
- $t = k$ (accepting?): delete every $b_{k,j,\delta}$ whose state q is *not* accepting

$\rightsquigarrow$ $|V| \le (k+1)(k+2) \cdot (|\Sigma| + 2 + |\Delta|)$ (polytime ✓); parameter $k' = (k+1)(k+2) = g(k)$ ✓

(size of transition table)

$|V| = n^{O(1)}$

p-Independent-Set is W[1]-complete – Rows and Columns

Groups of vertices:

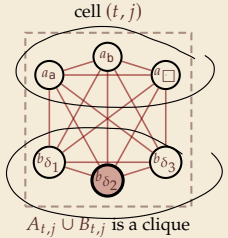
$A_{t,j}$ = all $a_{t,j,\circ}$

$B_{t,j}$ = all $b_{t,j,\circ}$

$B_{t,j,q,\sigma}$ = all $b_{t,j,(q,\sigma,\circ,\circ,\circ)}$

$B_{t,j,\circ,\sigma} = \bigcup_q B_{t,j,q,\sigma}$

$B_{t,j,q,\circ} = \bigcup_\sigma B_{t,j,q,\sigma}$



p-Independent-Set is W[1]-complete – Rows and Columns

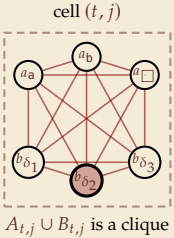
Groups of vertices: $A_{t,j}$ = all $a_{t,j,\circ}$

$B_{t,j}$ = all $b_{t,j,\circ}$

$B_{t,j,q,\sigma}$ = all $b_{t,j,(q,\sigma,\circ,\circ,\circ)}$

$B_{t,j,\circ,\sigma} = \bigcup_q B_{t,j,q,\sigma}$

$B_{t,j,q,\circ} = \bigcup_\sigma B_{t,j,q,\sigma}$



Edges = incompatibilities. vertices connected iff cannot both be "true" in one computation.

(I) **inside cliques:** every $A_{t,j} \cup B_{t,j}$ is a clique

$\rightsquigarrow$ an IS has ≤ 1 vertex per cell;

with $|I| = k' = (k+1)(k+2)$ cells: *exactly* one

p-Independent-Set is W[1]-complete – Rows and Columns

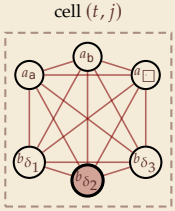
Groups of vertices: $A_{t,j}$ = all $a_{t,j,\circ}$

$B_{t,j}$ = all $b_{t,j,\circ}$

$B_{t,j,q,\sigma}$ = all $b_{t,j,(q,\sigma,\circ,\circ,\circ)}$

$B_{t,j,\circ,\sigma} = \bigcup_q B_{t,j,q,\sigma}$

$B_{t,j,q,\circ} = \bigcup_\sigma B_{t,j,q,\sigma}$



Edges = incompatibilities. vertices connected iff cannot both be "true" in one computation.

(I) **inside cliques:** every $A_{t,j} \cup B_{t,j}$ is a clique

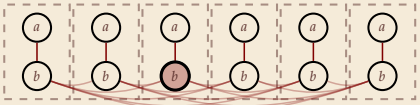
$\rightsquigarrow$ an IS has ≤ 1 vertex per cell;

with $|I| = k' = (k+1)(k+2)$ cells: *exactly* one

(R) **row cliques:** $\bigcup_j B_{t,j}$ is a clique for every t

$\rightsquigarrow$ at most one head position per row;

(at least one follows from (iv) on the next slide)



row i: (i) vertical, (ii) the b's pairwise

p-Independent-Set is W[1]-complete – Transitions

(S) **symbols persist where the head is not:**

$a_{t,j,\sigma}$ adjacent to all vertices in $(t+1, j)$

except $a_{t+1,j,\sigma}$ and $B_{t+1,j,\circ,\sigma}$

$\rightsquigarrow$ cell j still contains σ after step $t+1$

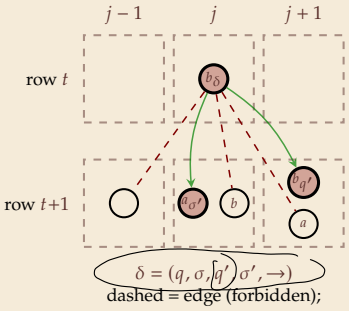
p-Independent-Set is W[1]-complete – Transitions

(S) **symbols persist where the head is not:**
$a_{t,j,\sigma}$ adjacent to all vertices in $(t+1, j)$
except $a_{t+1,j,\circlearrowleft}$ and $B_{t+1,j,\circ,\circlearrowleft}$
$\rightsquigarrow$ cell j still contains σ after step $t+1$

(H) **head executes** $\delta = (q, \sigma, q', \sigma', d)$**:**
$b_{t,j,\delta}$ is adjacent to every vertex of

- $(t+1, j)$ except $a_{t+1,j,\sigma'}$ (write σ')
- $(t+1, j+d)$ except $B_{t+1,j+d,q',\circ}$ (move, state q')

(for $d = 0$: column j except $B_{t+1,j,q',\sigma'}$;
$d = -1$ never applies at $j = 0$ since \$ is not rewritten)

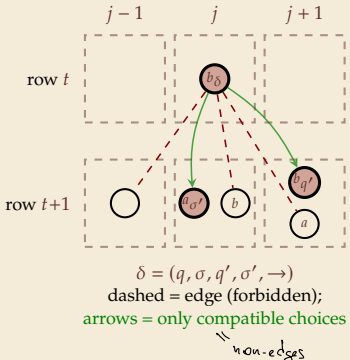


p-Independent-Set is W[1]-complete – Transitions

(S) **symbols persist where the head is not:**
$a_{t,j,\sigma}$ adjacent to all vertices in $(t+1, j)$
except $a_{t+1,j,\sigma}$ and $B_{t+1,j,\circ,\sigma}$
$\rightsquigarrow$ cell j still contains σ after step $t+1$

(H) **head executes** $\delta = (q, \sigma, q', \sigma', d)$**:**
$b_{t,j,\delta}$ is adjacent to every vertex of
- $(t+1, j)$ except $a_{t+1,j,\sigma'}$ (write σ')
- $(t+1, j+d)$ except $B_{t+1,j+d,q',\circ}$ (move, state q')

(for $d = 0$: column j except $B_{t+1,j,q',\sigma'}$;
$d = -1$ never applies at $j = 0$ since \$ is not rewritten)



$\rightsquigarrow$ Every $b^t_{\ell,j,\delta}$ is joined to *all* a-vertices of the target column in row $t+1$
$\rightsquigarrow$ the cell under the new head position must choose a b-vertex
$\rightsquigarrow$ a head exists in row $t+1$, with state q'.

p-Independent-Set is W[1]-complete – Correctness

Claim: M accepts x within k steps $\iff$ G has an IS of size $k' = (k+1)(k+2)$.

p-Independent-Set is W[1]-complete – Correctness

Claim: M accepts x within k steps $\iff$ G has an IS of size $k' = (k+1)(k+2)$.

"$\Rightarrow$" $\exists$ accepting computation $C_0, \ldots, C_k$ $\rightsquigarrow$ select, per cell, the vertex describing it

(for the head cell: the transition used next; in row k: an accepting-state b exists since accepting states are absorbing).

By construction no two selected vertices are incompatible $\rightsquigarrow$ independent set.

in row t pick $b_{t,j,\delta}$ for the position j of head in C_t and δ the transition used from $C_t \to C_{t+1}$

for all $j' \neq j$ pick $a_{t,j',\sigma}$ for σ the j'th tape letter in C_t

p-Independent-Set is W[1]-complete – Correctness

Claim: M accepts x within k steps $\iff$ G has an IS of size $k' = (k+1)(k+2)$.

"$\Rightarrow$" $\exists$ accepting computation $C_0, \ldots, C_k$ $\rightsquigarrow$ select, per cell, the vertex describing it
(for the head cell: the transition used next; in row k: an accepting-state b exists since accepting states are absorbing).
By construction no two selected vertices are incompatible $\rightsquigarrow$ independent set.

"$\Leftarrow$" let I be independent, $|I| = k'$. (I) $\rightsquigarrow$ exactly one vertex per cell;
boundary deletions $\rightsquigarrow$ row 0 is the initial configuration.

p-Independent-Set is W[1]-complete – Correctness

Claim: M accepts x within k steps $\iff$ G has an IS of size $k' = (k+1)(k+2)$.

"$\Rightarrow$" $\exists$ accepting computation $C_0, \ldots, C_k$ $\rightsquigarrow$ select, per cell, the vertex describing it
(for the head cell: the transition used next; in row k: an accepting-state b exists since accepting states are absorbing).
By construction no two selected vertices are incompatible $\rightsquigarrow$ independent set.

"$\Leftarrow$" let I be independent, $|I| = k'$. (I) $\rightsquigarrow$ exactly one vertex per cell;
boundary deletions $\rightsquigarrow$ row 0 is the initial configuration.
Induction on t: row t describes a config C_t, exactly one head;
(R) + (H) $\rightsquigarrow$ row $t+1$ has exactly one head,
(S) + (H) $\rightsquigarrow$ C_{t+1} follows from C_t by transition δ chosen in row t.

p-Independent-Set is W[1]-complete – Correctness

Claim: M accepts x within k steps $\iff$ G has an IS of size $k' = (k+1)(k+2)$.

"$\Rightarrow$" $\exists$ accepting computation $C_0, \ldots, C_k$ $\rightsquigarrow$ select, per cell, the vertex describing it
(for the head cell: the transition used next; in row k: an accepting-state b exists since accepting states are absorbing).
By construction no two selected vertices are incompatible $\rightsquigarrow$ independent set.

"$\Leftarrow$" let I be independent, $|I| = k'$. (I) $\rightsquigarrow$ exactly one vertex per cell;
boundary deletions $\rightsquigarrow$ row 0 is the initial configuration.
Induction on t: row t describes a config C_t, exactly one head;
(R) + (H) $\rightsquigarrow$ row $t+1$ has exactly one head,
(S) + (H) $\rightsquigarrow$ C_{t+1} follows from C_t by transition δ chosen in row t.
Row k: head vertex has an accepting state $\rightsquigarrow$ M accepts after k steps.

p-Independent-Set is W[1]-complete – Correctness

Claim: M accepts x within k steps $\iff$ G has an IS of size $k' = (k+1)(k+2)$.

"$\Rightarrow$" $\exists$ accepting computation $C_0, \ldots, C_k$ $\rightsquigarrow$ select, per cell, the vertex describing it
(for the head cell: the transition used next; in row k: an accepting-state b exists since accepting states are absorbing).
By construction no two selected vertices are incompatible $\rightsquigarrow$ independent set.

"$\Leftarrow$" let I be independent, $|I| = k'$. (I) $\rightsquigarrow$ exactly one vertex per cell;
boundary deletions $\rightsquigarrow$ row 0 is the initial configuration.
Induction on t: row t describes a config C_t, exactly one head;
(R) + (H) $\rightsquigarrow$ row $t+1$ has exactly one head,
(S) + (H) $\rightsquigarrow$ C_{t+1} follows from C_t by transition δ chosen in row t.
Row k: head vertex has an accepting state $\rightsquigarrow$ M accepts after k steps.

- polytime construction, $k' = (k+1)(k+2)$ depends on k only $\rightsquigarrow$ fpt-reduction ■

Clicker Question



The above construction can simulate any nondeterministic TM. Does this prove that IndependentSet is NP-hard without using Cook-Levin (SAT NP-hard)?

A Yes!

B No

C No idea.



→ *sli.do/advalg*

Clicker Question



The above construction can simulate any nondeterministic TM. Does this prove that IndependentSet is NP-hard without using Cook-Levin (SAT NP-hard)?

- **A** Yes! ✓
- **B** ~~No~~
- **C** ~~No idea.~~

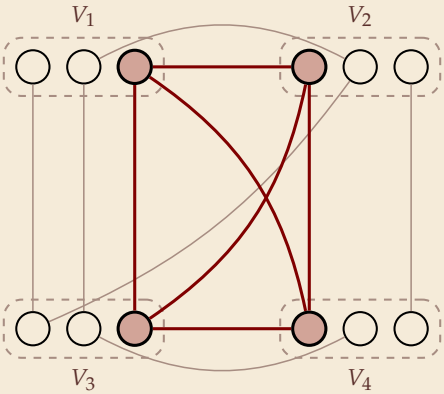


→ *sli.do/advalg*

Multicolored Clique – the Workhorse of W[1]-Hardness

Definition 5.15 (p-Multicolored-Clique)

Given: graph $G = (V, E)$, $k \in \mathbb{N}$, and a partition $V = V_1 \dot\cup \cdots \dot\cup V_k$ ("color classes")
Parameter: k
Question: Is there a clique with exactly one vertex from each V_i?



Multicolored Clique – Hardness

Theorem 5.16
p-Multicolored-Clique is W[1]-complete.

Proof:

$\in$ W[1]: guess one vertex per class (k guesses), check all $\binom{k}{2}$ pairs
$\rightsquigarrow$ tail-nondeterministic, κ-restricted. NRAM

Multicolored Clique – Hardness

Theorem 5.16
p-Multicolored-Clique is W[1]-complete. ◀

Proof:

$\in$ W[1]: guess one vertex per class (k guesses), check all $\binom{k}{2}$ pairs
$\rightsquigarrow$ tail-nondeterministic, κ-restricted.

W[1]-hard: p-Clique $\leq_{fpt}$ p-Multicolored-Clique. Given (G, k), take k copies of V:

$$V_i = V \times \{i\}, \qquad E' = \big\{\{(u,i),(v,j)\} : i \neq j,\ \{u,v\} \in E\big\}.$$

Multicolored Clique – Hardness

Theorem 5.16
p-Multicolored-Clique is W[1]-complete. ◀

Proof:

$\in$ W[1]: guess one vertex per class (k guesses), check all $\binom{k}{2}$ pairs
$\rightsquigarrow$ tail-nondeterministic, κ-restricted.

W[1]-hard: p-Clique $\leq_{fpt}$ p-Multicolored-Clique. Given (G, k), take k copies of V:

$$V_i = V \times \{i\}, \qquad E' = \big\{\{(u,i),(v,j)\} : i \neq j,\ \{u,v\} \in E\big\}.$$

Equivalence $\{(u,i),(v,j)\} \notin E'$

"$\Leftarrow$" A multicolored clique $(v_1, 1), \ldots, (v_k, k)$ has pairwise adjacent v_i
hence $v_i \neq v_j$ (no loops in G)
$\rightsquigarrow$ a k-clique in G

Multicolored Clique – Hardness

Theorem 5.16

p-Multicolored-Clique is W[1]-complete. ◀

Proof:

$\in$ W[1]: guess one vertex per class (k guesses), check all $\binom{k}{2}$ pairs

$\rightsquigarrow$ tail-nondeterministic, κ-restricted.

W[1]-hard: p-Clique $\leq_{fpt}$ p-Multicolored-Clique. Given (G, k), take k copies of V:

$$V_i = V \times \{i\}, \qquad E' = \big\{\{(u,i),(v,j)\} : i \neq j,\ \{u,v\} \in E\big\}.$$

Equivalence

"$\Leftarrow$" A multicolored clique $(v_1, 1), \ldots, (v_k, k)$ has pairwise adjacent v_i
hence $v_i \neq v_j$ (no loops in G)
$\rightsquigarrow$ a k-clique in G

"$\Rightarrow$" A k-clique $\{v_1, \ldots, v_k\}$ in G gives the multicolored clique (v_i, i)

Polytime, $k' = k$. ■

Example: Dominating Set is W[1]-hard

Why colours? Reductions *from* Multicolored Clique can build

- **class gadgets** ("which vertex of V_i?") and
- **pair-of-classes gadgets** ("are the two choices adjacent?")

Theorem 5.18

p-Dominating-Set is W[1]-hard.

Proof:

We show p-Multicolored-Clique $\leq_{fpt}$ p-Dominating-Set.

Example: Dominating Set is W[1]-hard

Why colours? Reductions *from* Multicolored Clique can build

- **class gadgets** ("which vertex of V_i?") and
- **pair-of-classes gadgets** ("are the two choices adjacent?")

Theorem 5.18
p-Dominating-Set is W[1]-hard.

Proof:

We show p-Multicolored-Clique $\leq_{fpt}$ p-Dominating-Set.
Given $(G, k, V_1, \ldots, V_k)$, build H:

- **class gadget**, $\forall i$:
  make V_i clique + new vertices x_i, y_i
  adjacent to all of V_i (and nothing else).

Example: Dominating Set is W[1]-hard

Why colours? Reductions *from* Multicolored Clique can build

- **class gadgets** ("which vertex of V_i?") and
- **pair-of-classes gadgets** ("are the two choices adjacent?")

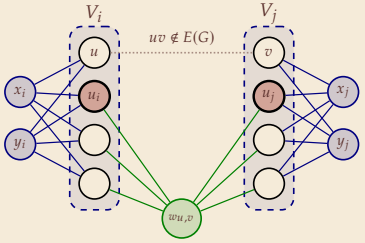
Theorem 5.18

p-Dominating-Set is W[1]-hard.

Proof:

We show p-Multicolored-Clique $\leq_{fpt}$ p-Dominating-Set.
Given $(G, k, V_1, \ldots, V_k)$, build H:

- **class gadget**, $\forall i$:
  make V_i clique + new vertices x_i, y_i
  adjacent to all of V_i (and nothing else).
- **pair gadget**, $\forall i < j$ and **non-edge**
  $\{u, v\} \notin E$ with $u \in V_i$, $v \in V_j$:
  new $w_{u,v}$ adjacent to $V_i \setminus \{u\}$ and $V_j \setminus \{v\}$.



u_i, u_j dominate $w_{u,v}$; choosing u and v would not.
tinted box = clique in H; edges of G are *not* copied into H

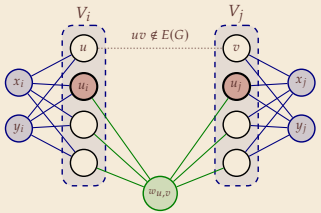
Example: Dominating Set is W[1]-hard [2]

Proof (continued):

Target size $k' = k$.

Polytime: $|V(H)| \leq n + 2k + n^2$.

Claim: G has a multicolored clique $\iff$ H has a dominating set of size k.



u_i, u_j dominate $w_{u,v}$; choosing u and v would not.
tinted box = clique in H; edges of G are *not* copied into H

Example: Dominating Set is W[1]-hard [2]

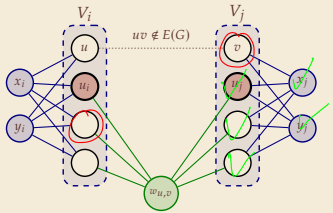
Proof (continued):

Target size $k' = k$.

Polytime: $|V(H)| \leq n + 2k + n^2$.

Claim: G has a multicolored clique $\iff$ H has a dominating set of size k.

"$\Rightarrow$" $u_1 \in V_1, \ldots, u_k \in V_k$ multicolored clique.
u_i dominates $V_i \cup \{x_i, y_i\}$ (cliques).



u_i, u_j dominate $w_{u,v}$; choosing u and v would not.
tinted box = clique in H; edges of G are *not* copied into H

Example: Dominating Set is W[1]-hard [2]

Proof (continued):

Target size $k' = k$.

Polytime: $|V(H)| \leq n + 2k + n^2$.

Claim: G has a multicolored clique $\iff$ H has a dominating set of size k.

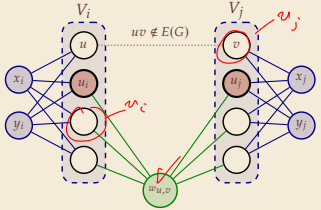
"$\Rightarrow$" $u_1 \in V_1, \ldots, u_k \in V_k$ multicolored clique.

u_i dominates $V_i \cup \{x_i, y_i\}$ (cliques).

$w_{u,v}$ ($u \in V_i$, $v \in V_j$ non-adjacent) dominated:

$(u, v) \neq (u_i, u_j)$ since $u_i u_j \in E$ (clique!), so

$u_i \in V_i \setminus \{u\}$ or $u_j \in V_j \setminus \{v\}$.



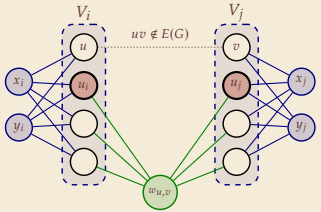
u_i, u_j dominate $w_{u,v}$; choosing u and v would not.
tinted box = clique in H; edges of G are *not* copied into H

Example: Dominating Set is W[1]-hard [2]

Proof (continued):

Target size $k' = k$.
Polytime: $|V(H)| \leq n + 2k + n^2$.

Claim: G has a multicolored clique $\iff$ H has a dominating set of size k.



u_i, u_j dominate $w_{u,v}$; choosing u and v would not.
tinted box = clique in H; edges of G are *not* copied into H

"$\Rightarrow$" $u_1 \in V_1, \ldots, u_k \in V_k$ multicolored clique.
u_i dominates $V_i \cup \{x_i, y_i\}$ (cliques).

$w_{u,v}$ ($u \in V_i$, $v \in V_j$ non-adjacent) dominated:
$(u, v) \neq (u_i, u_j)$ since $u_i u_j \in E$ (clique!), so
$u_i \in V_i \setminus \{u\}$ or $u_j \in V_j \setminus \{v\}$.

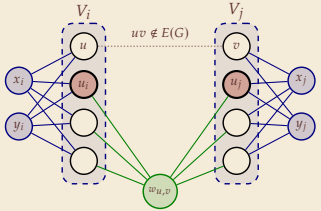
"$\Leftarrow$" Let D dominate H, $|D| = k$.
Closed neighbourhoods $\bar{N}(x_i)$ and $\bar{N}(y_i)$ lie in disjoint sets $V_i \cup \{x_i, y_i\}$, so D has *exactly one* vertex d_i in each of them. d_i must dominate both x_i and y_i $\rightsquigarrow$ $d_i \in V_i$.
(This is what y_i is for; D contains no w-vertices.)

Example: Dominating Set is W[1]-hard [2]

Proof (continued):

Target size $k' = k$.
Polytime: $|V(H)| \leq n + 2k + n^2$.

Claim: G has a multicolored clique $\iff$ H has a dominating set of size k.



u_i, u_j dominate $w_{u,v}$; choosing u and v would not.
tinted box = clique in H; edges of G are *not* copied into H

"$\Rightarrow$" $u_1 \in V_1, \ldots, u_k \in V_k$ multicolored clique.
u_i dominates $V_i \cup \{x_i, y_i\}$ (cliques).

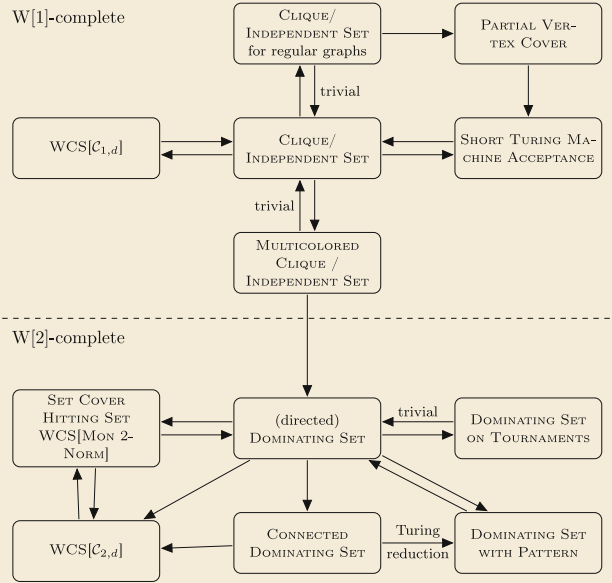
$w_{u,v}$ ($u \in V_i$, $v \in V_j$ non-adjacent) dominated:
$(u, v) \neq (u_i, u_j)$ since $u_i u_j \in E$ (clique!), so
$u_i \in V_i \setminus \{u\}$ or $u_j \in V_j \setminus \{v\}$.

"$\Leftarrow$" Let D dominate H, $|D| = k$.
Closed neighbourhoods $\bar{N}(x_i)$ and $\bar{N}(y_i)$ lie in disjoint sets $V_i \cup \{x_i, y_i\}$, so D has *exactly one* vertex d_i in each of them. d_i must dominate both x_i and y_i $\rightsquigarrow$ $d_i \in V_i$.

(This is what y_i is for; D contains no w-vertices.)

If $d_i d_j \notin E$, then w_{d_i,d_j} exists and none of its neighbours $(V_i \setminus \{d_i\}, V_j \setminus \{d_j\})$ is in D ↯
$\rightsquigarrow$ $d_1, \ldots, d_k$ pairwise adjacent $\rightsquigarrow$ multicolored clique. ■

Cygan et al. Reduction Network



adapted from **Fig. 13.4** of *Cygan et al.* (2015)

Conclusion

- some care is needed to lift complexity theory to parameterized problems
- but: theory of W[1]-hardness and fpt-reductions is an effective framework to show that a parameterized problem is unlikely to admit an fpt algorithm
  - $\mathsf{W}[1] \supsetneq \mathsf{FPT}$ widely believed, but *open*
    (standard evidence was "FPT = W[1] would refute ETH"; Oct 2026: ETH refuted, so this argument is void – see next unit)
  - need new "gadgets" for fpt reductions
- further refinements possible (W[t] hierarchy)
  - p-DominatingSet is W[1]-hard, but likely $\notin \mathsf{W}[1]$.
    (can be shown to be W[2]-complete and likely $\mathsf{W}[2] \supsetneq \mathsf{W}[1]$)
- W[1]-hardness suffices for negative results